PROJECT ASSIGNMENT

ClassTrack

Smart Mobile Attendance and Quiz System

Field	Details
Course	Mobile Programming – Level 2
Instructor	Dr. Amany
Technology Stack	Flutter (Dart), Firebase (Auth, Firestore, Storage)
Project Duration	4 weeks (one month)
Team Size	Groups of 4–5 students
Deliverable Type	Working mobile application + source code + short demo

1. Project Overview

Manual, paper-based attendance is slow and easy to abuse: a student can mark a classmate as "present" even when that classmate is not actually in the room. ClassTrack is a Flutter mobile application that solves this problem by letting a teacher open a short, location-verified attendance session that only students who are physically present in the classroom can complete.

Each group will design and build a complete Flutter + Firebase mobile application that supports two attendance modes, described in Section 3. The application must include both a teacher experience and a student experience within a single app (role selected at login).

Project Scenario

A university wants to reduce manual attendance-taking and prevent students from registering attendance when they are not physically present in the classroom. The system must allow a teacher to start a short attendance session and must support two attendance mechanisms:

- Mode 1 – Location Punch-In: the teacher starts a 2-minute attendance session; enrolled students attempt to register attendance; the system verifies that each student is within the permitted geographical radius before recording attendance.
- Mode 2 – Location-Restricted Quiz: the teacher starts a 2-minute quiz session containing five questions; only students who satisfy the location requirement may participate; the system records their answers, score, submission time, and attendance status.

2. User Roles and Responsibilities

Teacher

- Creates and manages class groups.
- Registers students, or shares a class join code that students use to enroll themselves.
- Starts an attendance session and selects its mode (Punch-In or Quiz).

- Defines the allowed geographical radius for the session.
- Monitors, in real time, which students have successfully attended.

Student

- Joins a class using the class join code.
- Sees active attendance sessions for classes they are enrolled in.
- Grants the application location access.
- Punches in if physically located inside the permitted area.
- In Mode 2, answers and submits the quiz before the session expires.

3. Functional Requirements

3.1 Mode 1 — Location-Based Punch-In

The teacher starts a punch-in request from within the app. The request stays valid for exactly 2 minutes. During that window, any enrolled student whose device location is within the teacher's configured radius may punch in and be recorded as present.

Example Walkthrough

The teacher is physically in the classroom and presses START ATTENDANCE. The application records a new session:

```
Session ID:      ATT-2026-0922-001
Teacher:         Dr. Amany
Class:            Mobile Programming Level 2
Location:        30.0444, 31.2357
Allowed Radius:  50 m
Start Time:      09:00:00
Expiration:      09:02:00
```

Enrolled students immediately see the following on their device:

```
Attendance Session Active
Mobile Programming
Valid for: 01:32
Check your location to punch in.
```

When a student presses PUNCH IN, the application (server-side) evaluates:

```
Distance(Student, Teacher) <= Allowed Radius ?
```

Accepted example: Distance = 23 m, Allowed = 50 m → Attendance accepted.

Rejected example: Distance = 183 m, Allowed = 50 m → Attendance rejected.

3.2 Mode 2 — Location-Restricted Quiz Attendance

Mode 2 extends Mode 1. Instead of a single PUNCH IN button, the session becomes a short, timed quiz that a student must complete correctly (i.e., submit) while inside the permitted area.

```
Attendance Quiz
5 questions
Time: 2 minutes
```

To be marked present in Mode 2, a student must:

1. Be enrolled in the class.

2. Be within the permitted geographical area.
3. Open the quiz.
4. Answer all 5 questions.
5. Submit before the session expires.

For every attempt, the system must record: student, class, session ID, whether location was verified, whether the quiz was submitted, submission time, score, and final attendance status.

Sample Question

```
Question 1
What protocol is used by HTTP?
A. TCP
B. UDP
C. ICMP
D. ARP
```

Example Session Results

Student	Distance	Quiz	Score	Attendance
Ahmed	18 m	Submitted	4/5	Present
Sara	31 m	Submitted	5/5	Present
Mohamed	124 m	Submitted	5/5	Rejected
Ali	22 m	Not submitted	—	Absent

Quiz Content Source (Required Approach)

Each group must build the five quiz questions inside the Flutter application itself and store them in Firestore (multiple-choice, single correct answer). This gives every group hands-on practice with Flutter forms, radio-button widgets, Firestore reads/writes, input validation, countdown timers, and score calculation. A Google Forms-based quiz (the teacher supplies a Google Form URL and the app opens or embeds it) is available only as an optional bonus feature — see Section 7 — because it introduces Google API authentication and response-retrieval complexity that is not required for this course.

4. Notification and Location-Validation Rule

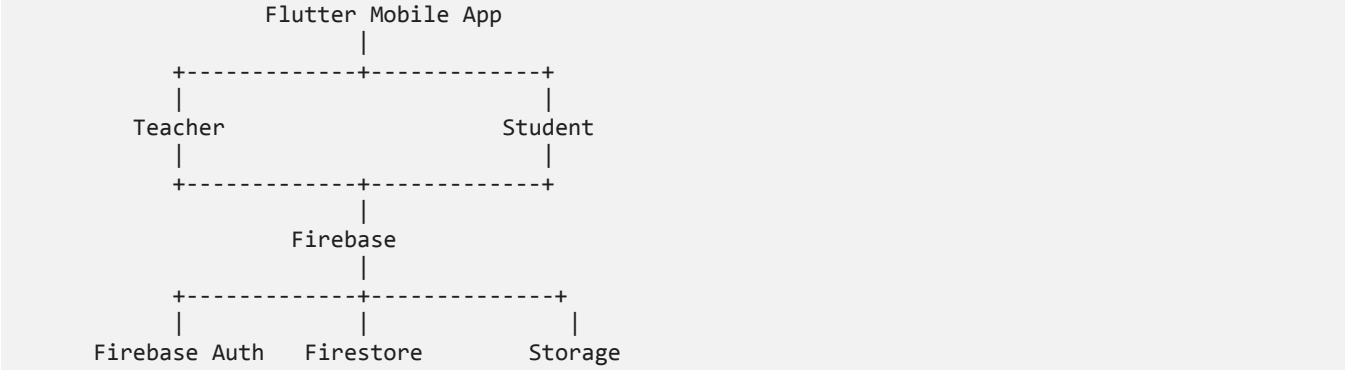
This rule is the single most important design clarification in the assignment, and it must be followed exactly:

All enrolled students may see that an attendance session is active, regardless of their location. Only students whose verified device location falls within the permitted radius may successfully punch in (Mode 1) or have their quiz submission accepted (Mode 2). A student outside the radius sees the session but receives a clear rejection message when attempting to punch in or submit.

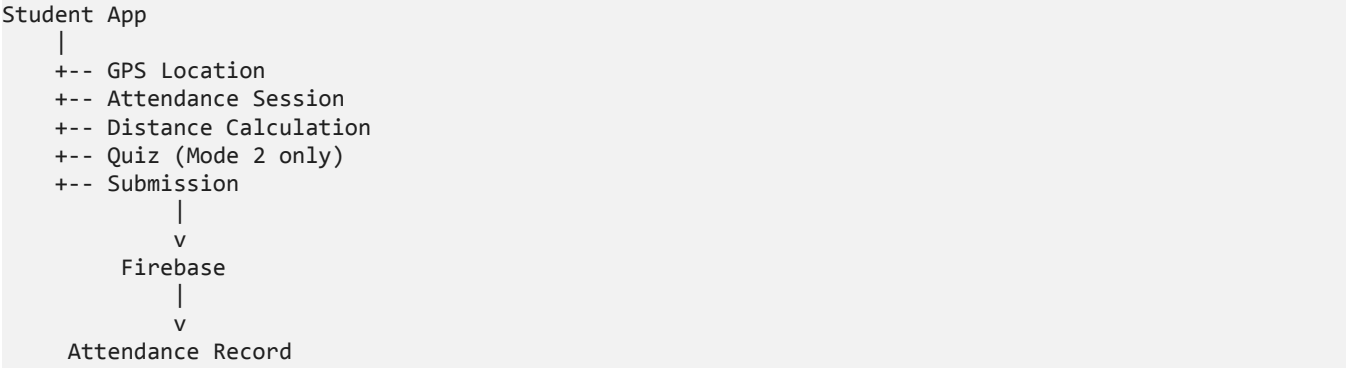
Do not implement a requirement where students outside the radius cannot see the notification at all. Detecting a student's location while the app is closed or backgrounded requires background location access, OS-specific permission handling, and battery-restriction workarounds that are beyond the scope of a one-month, Level-2 course project. Visibility of the session is unrestricted; only the ability to be recorded as present is location-gated, and that check must be enforced in the backend (Firestore rules or a Cloud Function), not only in the Flutter client, so that a modified client cannot bypass it.

5. System Architecture

Your application must follow this high-level architecture:



Student-side data flow for a single attendance attempt:



6. Firebase Database Design

Use the following Firestore structure as your baseline schema. You may add fields, but do not remove the ones listed below — later sections of the course assume they exist.

users

Field	Description
name	Full name of the user
email	Login email
role	"teacher" or "student"
studentID	University/student ID (students only)

classes

Field	Description
name	Class name
teacherID	Owning teacher's user ID

semester	Semester/term label

enrollments

Field	Description
classID	Reference to classes
studentID	Reference to users

attendance_sessions

Field	Description
classID	Reference to classes
teacherID	Reference to users
mode	"punch_in" or "quiz"
latitude / longitude	Teacher's location when the session was started
radius	Allowed radius in meters
startTime / expiryTime	Session validity window (2 minutes)
status	"active" or "closed"

attendance_records

Field	Description
sessionID	Reference to attendance_sessions
studentID	Reference to users
timestamp	When the attempt was made
latitude / longitude	Student's location at attempt time
distance	Computed distance from teacher, in meters
status	"present", "rejected", or "absent"
quizScore	Score out of 5 (Mode 2 only)

7. Attendance Validation Algorithm

An attendance attempt is accepted only if all four conditions below are true:

```
Current Time  <  Expiry Time
        AND
Student belongs to class
        AND
Distance  <=  Allowed Radius
        AND
Student has not already attended this session
```

Implement this as a sequential decision process:

```
Student requests attendance
        |
        v
   Is session active?  --NO--> Reject
        | YES
        v
   Is student enrolled?  --NO--> Reject
        | YES
        v
   Check GPS available?  --NO--> Reject
        | YES
        v
   Within radius?  --NO--> Reject
        | YES
        v
      Accept
```

8. Four-Week Development Plan

Week	Focus	Tasks
Week 1	Foundation	Flutter project setup; Firebase connection; login/register; teacher/student roles; basic navigation; class creation; student joining a class.
Week 2	Mode 1	Teacher starts attendance; session generation; 2-minute timer; GPS acquisition; distance calculation; radius validation; punch-in; Firestore attendance record.
Week 3	Mode 2	Five in-app questions; quiz interface; countdown timer; answer validation; submission; score calculation; attendance based on successful, in-range submission.
Week 4	Management & Testing	Teacher dashboard (live session view); attendance history; session history; quiz results; basic reports; error handling; UI polish; testing; final presentation/demo.

Example teacher dashboard view for a closed session:

```
Class: Mobile Programming
Attendance Session
------------------
Ahmed       Present
Sara        Present
Mohamed     Absent
Ali         Present
```

9. Mandatory Requirements

Every group's submission must satisfy all of the following:

- Teacher and student login/registration.
- Class creation and joining (via join code or teacher registration).
- Teacher can start an attendance session in either mode.

- Session validity is exactly 2 minutes.
- GPS location verification for every attempt.
- Configurable radius, set by the teacher at session start.
- At most one attendance record per student per session.
- Mode 1: single-tap punch-in flow.
- Mode 2: five-question in-app quiz flow.
- Teacher-facing attendance/quiz results view.
- Firebase (Auth + Firestore) as the backend.
- Backend-side (not client-only) validation of session time, enrollment, and radius.

10. Optional Bonus Features

These are not required. Implement them only after all mandatory requirements are complete and tested.

- Push notifications for new attendance sessions.
- Google Forms integration as an alternative quiz source (Option B from Section 3.2).
- Attendance history and exportable CSV/PDF reports.
- Basic GPS-spoofing / mock-location detection.
- QR code as an alternative attendance mechanism.

11. Explicitly Out of Scope

The following are not required for this project and should not be attempted within the one-month timeframe. Groups may list any of these as “future work” in their final report.

- Facial recognition
- Fingerprint authentication
- Bluetooth beacons
- Wi-Fi fingerprinting
- Continuous background GPS tracking
- Complex geofencing
- AI-based cheating detection
- Advanced anti-GPS-spoofing mechanisms
- Full Google Workspace integration
- A complex administrator portal
- Multi-university / multi-semester architecture

12. Security and Data Integrity Note

A student could try to manipulate their device's reported latitude/longitude, or use a mock-location app. For this reason, the Flutter client must never be the final authority on whether a student is present. All validation — session status, enrollment, elapsed time, and distance — must be re-checked on the backend (Firestore security

rules or a Cloud Function) before an attendance record is written. Treat the pattern “if (distance < 50) print('present')” evaluated only on the phone as insecure and unacceptable for the final submission.

13. Submission Guidelines

Each group (4–5 students) must submit:

- Full Flutter source code (as a Git repository or a zipped project folder).
- The Firebase project structure used (a Firestore export or a schema description is sufficient).
- A short README describing setup steps, group member names/IDs, and each member's contribution.
- A short live or recorded demo showing both Mode 1 and Mode 2 working end to end.

Exact submission dates and any additional grading details will be announced separately by the instructor.